

# CONTRIBUTOR HANDBOOK

*Your guide to contributing meaningfully to open source*

---

## **Welcome to the event.**

This handbook will walk you through everything you need to know to participate confidently, contribute meaningfully, and grow as a developer.

# What Is This Event?

---

This event is a beginner-friendly open source contribution program inspired by initiatives like Hacktoberfest, GSSoC, and SWOC. It is designed to give you hands-on experience with real GitHub workflows in a safe and supportive environment.

The goal is NOT just to rack up pull requests. The goal is to:

- learn how real software projects are structured
- practice branching, commits, and pull requests
- understand how code review works
- build habits that will serve you in professional software development
- collaborate with others on shared codebases

Whether this is your first time using GitHub or your fiftieth contribution, this event has something for you.

# What You Will Learn

---

- How to navigate an unfamiliar codebase
- How to pick up and work on GitHub issues
- How to create branches and follow naming conventions
- How to write clear, professional commit messages
- How to raise a pull request and link it to an issue
- How to respond to code review feedback
- How to communicate professionally with maintainers

# Getting Started

---

## Step 1 — Set Up Git and GitHub

Before anything else, make sure you have:

- a GitHub account at [github.com](https://github.com)
- Git installed on your machine
- a code editor such as VS Code

If you are new to GitHub, take 30 minutes to explore how repositories, issues, and pull requests work. That context will make everything else easier.

## Step 2 — Explore Available Projects

Browse the list of participating projects shared by the organizers. Look for:

- projects that match your tech stack or interest area
- repositories with beginner-friendly labels
- projects with good README and setup documentation

You are not limited to one project. You can contribute to multiple repositories throughout the event.

## Step 3 — Fork and Clone

Once you have chosen a project:

1. Fork the repository to your own GitHub account
2. Clone your fork locally using `git clone`
3. Add the original repository as an upstream remote
4. Follow the setup instructions in the README
5. Make sure the project runs correctly on your machine before starting any work

Never work directly on the main branch of your fork. Always create a new branch for each contribution.

# Contribution Workflow

---

Follow these steps for every contribution you make:

6. Browse open issues in the repository
7. Read the issue description carefully
8. Comment on the issue to request assignment — do not start work before being assigned
9. Wait for the maintainer to assign the issue to you
10. Create a new branch using the correct naming format
11. Work on the issue in your branch
12. Commit your changes with clear commit messages
13. Push your branch to your fork
14. Open a pull request and link it to the issue
15. Respond to any review feedback from the maintainer
16. Wait for your PR to be reviewed and merged

## Branch Naming Format

Use the format specified in the project's CONTRIBUTING.md. A common format is:

```
type/short-description
```

### Examples:

```
feat/add-dark-mode  
fix/navbar-overflow  
docs/update-readme
```

## Commit Message Format

Write commit messages that clearly describe what you did. A common convention:

```
type: short summary of change
```

### Examples:

```
feat: add dark mode toggle to navbar  
fix: resolve overflow on mobile nav  
docs: update installation steps in README
```

# Understanding Issues

---

## How to Read an Issue

A well-written issue will contain:

- a clear title describing the task
- a description of the current problem or feature request
- the expected behavior or outcome
- relevant files or components to look at
- acceptance criteria — what 'done' looks like
- a difficulty label

Read the entire issue before commenting. If anything is unclear, ask a specific question in the comments.

## Difficulty Labels

| Label    | What It Means                                                  |
|----------|----------------------------------------------------------------|
| Beginner | Minimal project understanding required. Good for first-timers. |
| Easy     | Basic understanding required. Small features or simple fixes.  |
| Medium   | Requires understanding of the project structure and logic.     |
| Hard     | Requires deep architectural understanding. Advanced work.      |

## Issue Assignment Rules

- Always comment on an issue to request assignment before starting work
- Do not work on issues that are already assigned to someone else
- You can hold one active issue at a time initially
- If you cannot complete an issue, comment and let the maintainer know so it can be reassigned
- Inactivity for 24-48 hours may result in the issue being reassigned

# Pull Requests

---

## Before Raising a PR

Make sure you have:

- tested your changes locally and confirmed they work
- followed the coding conventions of the project
- not included unrelated changes in your PR
- synced your branch with the latest upstream changes
- reviewed your own diff before submitting

## Writing a Good PR Description

Your PR should clearly communicate what you did and why. Use the PR template if one is provided. At minimum include:

- the issue number your PR resolves (use 'Closes #issue-number')
- a summary of changes made
- screenshots or recordings if the change is visual
- a description of how you tested the changes

### GOOD PR Title Examples

feat: add responsive dark mode toggle — closes #42

fix: resolve navbar overflow on mobile screens — closes #17

docs: add environment variable setup to README — closes #8

### BAD PR Title Examples

fixed stuff

changes

update

## Keep PRs Focused

Each pull request should address exactly one issue. Do not:

- bundle multiple unrelated fixes into one PR
- make sweeping formatting changes across files you did not touch
- refactor code outside the scope of your issue
- include debug code, console logs, or commented-out blocks

# Handling Code Review

---

## What to Expect

After you raise a PR, the maintainer will review your code. They may:

- approve and merge your PR
- request changes with specific feedback
- ask questions about your implementation
- suggest a different approach

This is a normal and healthy part of software development. Code review is not criticism of you as a person. It is feedback on the code.

## How to Respond to Feedback

- Read all comments before making changes
- Ask for clarification if a comment is unclear
- Make the requested changes in your existing branch
- Push your updates and the PR will automatically update
- Reply to each comment to let the reviewer know it has been addressed
- Do not open a new PR for the same issue

## Review Response Examples

### Good Responses to Review Feedback

Maintainer: "Please make this component responsive for smaller screens."

You: "Done! Added media queries for screens below 768px. Please take another look."

Maintainer: "Can you explain why you used this approach here?"

You: "I used `useEffect` here because the data fetch needs to run after render. Happy to switch to a different approach if preferred."

### Poor Responses to Review Feedback

Maintainer: "Please improve variable naming."

You: "ok" (and nothing changes)

Maintainer: "This has a bug on edge cases."

You: (opens a new PR instead of fixing the existing one)



# Code Quality Standards

---

## What Maintainers Expect

- code that actually solves the issue
- readable variable and function names
- consistent formatting matching the project style
- no unrelated modifications to other files
- no broken existing functionality
- no console.log or debug artifacts left behind

## AI-Assisted Code

AI tools may be allowed depending on the event policy. However:

- you must understand every line of code you submit
- maintainers may ask you to explain your implementation
- blindly pasting generated code without understanding it is not acceptable
- if you used AI assistance, make sure the output is correct and fits the project

The goal of this event is for you to learn. Submitting code you do not understand defeats that purpose.

# Communication Guidelines

---

## How to Ask Good Questions

When asking a maintainer for help or clarification:

- be specific about what you tried and what happened
- share relevant error messages or screenshots
- mention which file or line number you are looking at
- ask in the issue comments or the designated channel, not in DMs

### Good Question Example

"I am trying to implement the dark mode toggle from issue #42.

When I click the button, the state updates but the styles do not change.

I have added the class toggle in App.js line 47 but the CSS is not applying.

Could you point me toward where the theme variables are defined?"

### Poor Question Example

"it's not working, help"

## General Communication Etiquette

- be respectful and professional in all interactions
- be patient — maintainers are volunteers too
- thank reviewers for their feedback
- do not demand immediate responses
- do not spam issues or PRs with unnecessary comments

## What NOT to Do

---

### Spam and Low-Quality Contributions

The following will NOT be counted and may result in disqualification:

- meaningless whitespace or formatting-only changes
- changing random colors or content with no purpose
- unnecessary README edits just to get a PR
- duplicate contributions already submitted by someone else
- copying another contributor's code
- submitting code you do not understand
- working on an issue without being assigned

Maintainers are trained to identify and reject low-quality contributions. Quality always wins over quantity.

### Common Mistakes to Avoid

| Mistake                             | What to Do Instead                                |
|-------------------------------------|---------------------------------------------------|
| Starting work without assignment    | Comment and wait to be assigned first             |
| Submitting unrelated changes        | Keep PRs scoped to the issue only                 |
| Ignoring review feedback            | Address every comment before re-requesting review |
| Opening a new PR for the same issue | Push fixes to the existing PR branch              |
| Vague PR descriptions               | Explain what, why, and how in your PR             |
| Working on main branch              | Always create a dedicated feature branch          |
| Multiple issues at once (initially) | Finish one issue before picking up another        |

# Contributor Checklist

---

## Before You Start

- ✓ GitHub account created
- ✓ Git installed and configured
- ✓ Repository forked and cloned
- ✓ Project setup and running locally
- ✓ README and CONTRIBUTING.md read

## For Every Contribution

- ✓ Issue read and understood
- ✓ Assignment requested and confirmed
- ✓ New branch created with correct naming
- ✓ Changes tested locally
- ✓ PR description written clearly
- ✓ Issue linked in PR
- ✓ Screenshots added if visual change
- ✓ Review feedback addressed

# You are ready to contribute.

---

**Remember:**

*One meaningful contribution is worth more than ten spam PRs.*

Ask questions freely. Make mistakes. Learn from feedback.  
That is what this event is for.

**Good luck. Happy contributing.**